

SWDD-EMS-001

Team SoftDev-1

Employee Management System

Software Design Document

Names: Nick Gilreath, Arjun Baskar, Miguel Perez, Matarr Touray, Peter St Pierre Jr., Felipe Benavides

Section: 004

Workstation: n/a

Date: (04/24/2026)

TABLE OF CONTENTS

1.0	INTRODUCTION.....	3
1.1	<i>Purpose</i>	3
1.2	<i>Scope.....</i>	3
1.3	<i>Overview</i>	3
1.4	<i>Reference Material.....</i>	3
1.5	<i>Definitions and Acronyms</i>	3
2.0	SYSTEM OVERVIEW.....	4
3.0	SYSTEM ARCHITECTURE.....	4
3.1	<i>Architectural Design</i>	4
3.2	<i>Decomposition Description.....</i>	5
3.3	<i>Design Rationale</i>	5
4.0	DATA DESIGN.....	5
4.1	<i>Data Description.....</i>	5
4.2	<i>Data Dictionary</i>	5
	<i>Employee Table:</i>	5
	<i>Division Table:.....</i>	6
	<i>JobTitle Table:</i>	6
	<i>PayStatement Table:</i>	6
5.0	COMPONENT DESIGN	6
6.0	HUMAN INTERFACE DESIGN.....	8
6.1	<i>Overview of User Interface.....</i>	8
6.2	<i>Screen Images</i>	8
6.3	<i>Screen Objects and Actions.....</i>	11
7.0	REQUIREMENTS MATRIX	13

1.0 INTRODUCTION

1.1 Purpose

This software design software document describes the design of a basic Employee Management System for Company '2'. The design includes the system architecture, components, and data structures used to implement the required functionality.

1.2 Scope

This project involves the design and implementation of a basic Employee Management System for a small company, which has fewer than 20 full-time employees, named Company '2' and it will operate without user authentication. It will manage employee data within a MySQL database.

The system will allow users to search for employees by employee ID, exact full-name, or SSN. As well as update employee information and apply salary increases based on specific percentage ranges. The system will also generate reports of pay statement history, total monthly pay by job title, and total monthly pay by division. There will be a user interface that will interact with a database using JDBC. The goal of the project is to deliver a simple, functional, and efficient solution to managing employees.

1.3 Overview

This document is organized in such a way so that several sections of the document describe the Employee Management System. Section 2 will provide a general overview of the system's functionality, context, and design. Section 3 will describe the system architecture, including decomposition and design rationale. Section 4 describes the data design, including data structures and data dictionary. Section 5 lays out the component design and logic of the system. Section 6 describes the human interface design which includes user interaction and screen layouts. Section 7 will provide the requirements matrix, which will map the system components to functional requirements.

1.4 Reference Material

The following materials were used for reference in the development of this document:

- System description and project requirements for the EMS
- Course lecture notes
- MySQL documentation
- JDBC Documentation

1.5 Definitions and Acronyms

- EMS – Employee Management System
- SSN – Social Security Number (stored without dashes)
- EmpID – Unique identifier assigned to each employee
- JDBC – Java Database Connectivity, used to connect Java applications to the MySQL database

- MySQL – Relational database management system used to store employee data
- UX – User Experience, referring to the interaction between the user and the system
- UI – Text-based user interface
- UML – Unified Modeling Language, used for system diagrams
- CRUD – Create, Read, Update, Delete operations for database management

2.0 SYSTEM OVERVIEW

The Employee Management System is a simple software application designed for Company '2' to manage and maintain employee information using a MySQL database. The system is intended for use by an administrator who manually enters and updates employee data. The EMS provides core functionality such as updating and searching for employee records using employee ID, exact full-name, or SSN. It also allows for updating employee salaries based on specified percentage increases within defined salary ranges. In addition to data management, the system generates basic reports required by the company, including full-time employee information with pay statement history, total monthly pay by job title, and total monthly pay by division. The system will use a minimal user interface and will connect to the database using JDBC. No user authentication or authorization features are required. The overall goal is to provide a simple and efficient process for managing employee data for a small organization.

3.0 SYSTEM ARCHITECTURE

3.1 Architectural Design

The EMS follows a modular and layered architecture to separate responsibilities. The system has 3 main subsystems: the UI layer, the application logic layer, and the data access layer.

- UI Layer:
This layer provides minimal interface for the admin to interact with the system. The UI will be implemented using a console-based interface. The UI handles user input like searching for employees, requesting reports, etc. and display results back to the user.
- Data Access Layer (with minimal embedded business logic):
This layer has application logic built within the EmployeeDAO class along with other basic data access operations. This class handles user requests and applies the relevant business rules, like the calculation of salary increases. It also communicates directly with the MySQL database. It performs CRUD operations on employee records and retrieves the relevant information needed to display reports. While a separate business logic layer could improve the overall efficiency in the long-term given the data grows large, the current system keeps it simple given the small company and small scope of the project.
- Database (MySQL):
The MySQL database stores all of the data including employee information, job titles, salary details, pay history, and divisions.

3.2 Decomposition Description

The system is divided into 4 main components based on its functionality:

- Main (UI): Handles user input and displays results. It displays menu options, collects user input, and calls the appropriate methods in the data access layer to perform searching, updating, and adding employee operations.
- Employee: A data model class that represents a single employee record. It stores employee attributes including division name and job title name (retrieved via database joins) and provides a formatted toString() method.
- EmployeeDAO: Contains all database related operations. This class executes SQL queries to perform tasks such as searching for an employee, inserting a new employee, updating employee information, and generating new reports.
- DatabaseConnection: Manages the connection between the MySQL database using JDBC. It provides a reusable method for establishing a connection that is used by the EmployeeDAO class.

3.3 Design Rationale

The system uses a layered architecture as mentioned (UI, application logic/data access) to separate responsibilities and keep everything organized in the design. This will ensure the system is easier to maintain and update.

This approach allows changes to one part of the system without affecting other parts of the system. It also provides readability for the user.

A monolithic design could have been considered but as the system grows it would make management much harder.

4.0 DATA DESIGN

4.1 Data Description

The system stores all data in a MySQL database named 'employeeData'. The database contains four tables: employee, division, jobTitle, and payStatement. The employee table is the central entity and references both division and jobTitle through foreign keys (divisionID and jobTitleID). The payStatement table references the employee table through the empID foreign key, allowing each employee to have multiple pay records. All data is accessed and manipulated through JDBC using statements in the EmployeeDAO class.

4.2 Data Dictionary

Employee Table:

- empID (INT) – Unique identifier for each employee (Primary Key)
- firstName (VARCHAR) – Employee's first name
- lastName (VARCHAR) – Employee's last name
- SSN (VARCHAR) – Employee Social Security Number (no dashes)
- email (VARCHAR) – Employee email address

- phone (VARCHAR) – Employee phone number
- salary (DOUBLE) – Employee’s current salary
- jobTitleID (INT) – Foreign key referencing job title
- divisionID (INT) – Foreign key referencing division

Division Table:

- divisionID (INT) – Unique identifier for division (Primary key)
- divisionName (VARCHAR) – Name of the division

JobTitle Table:

- jobTitleID (INT) – Unique identifier for job title (Primary key)
- jobTitleName (VARCHAR) – Name of the job title

PayStatement Table:

- payID (INT) – Unique identifier for pay record (Primary key)
- empID (INT) – Foreign key referencing employee
- payDate (DATE) – Date of the pay period
- grossPay (DOUBLE) – Gross pay amount for the pay period
- netPay (DOUBLE) – Net pay amount for the pay period

5.0 COMPONENT DESIGN

Main.java (UI):

Purpose: Handle user interaction and menu navigation.

Logic:

- Display the menu in a loop
- Read user input
- Call EmployeeDAO methods based on selection

Operations:

- 1 → searchEmployee(searchTerm)
- 2 → addEmployee(...)
- 3 → updateEmployee(empID, email, phone)
- 4 → increaseSalary(min, max, percent)
- 5 → totalPayByDivision()
- 6 → getPayHistory(empID)
- 7 → updateEmployeeFull(empID, salary, jobTitleID, divisionID)
- 8 → totalPayByJobTitle()
- 9 → exit

Employee.java (Data Model):

Purpose: Represents an employee record with fields for ID, name, SSN, email, phone, salary,

division, and job title.

Includes a toString() method for formatted display.

EmployeeDAO.java (Data Access Layer):

Purpose: Handles all database operations using JDBC.

searchEmployee(searchTerm):

- Connect to database
- Execute SELECT with JOINs
- Match searchTerm against empID, SSN, or exact full-name
- Return a List of employee objects to Main.java
- (In Main.java): if a list is empty print "Employee not found." Else: loop through list and print each employee

addEmployee(...):

- Connect to database
- Prepare INSERT statement
- Set parameters
- Execute update

updateEmployee(empID, email, phone):

- Connect to database
- Prepare UPDATE statement
- Execute update

increaseSalary(min, max, percent):

- Connect to database
- Update salary = salary * (1 + percent)
- Apply condition: salary >= min AND salary < max
- Execute update

totalPayByDivision():

- Connect to database
- Execute GROUP BY query
- Print division totals

getPayHistory(empID):

- Connect to database
- Prepare SELECT statement with empID
- Execute query
- Display pay history records

updateEmployeeFull(empID, salary, jobTitleID, divisionID):

- Connect to database
- Prepare UPDATE statement
- Set parameters
- Execute update

totalPayByJobTitle():

- Connect to database
- Execute GROUP BY query on job title
- Calculate total pay
- Display results

DatabaseConnection.java:

Purpose: Establishes a MySQL connection.

Logic:

- Load JDBC
- Connect using URL, username, password
- Return connection object

6.0 HUMAN INTERFACE DESIGN

6.1 Overview of User Interface

The system displays a console-based user interface for an admin to interact with the EMS for efficiency and simplicity. The interface displays a menu of available actions and prompts the user to select an option from the list via a number entry.

The following actions can be performed from the interface:

- Search for an employee by ID, name, or SSN
- Add a new employee by entering required details
- Update an employee's email and phone number
- Apply a salary increase within a specified salary range and percentage
- Generate a report showing total pay by division
- View pay statement history for an employee
- Perform a full update of an employee's salary, job title, and division
- Generate a report showing total pay by job title
- Exit the system

For each option, the system requests the necessary input from the user and displays confirmation messages when applicable, errors, or retrieved data like employee data like their salary. All the outputs are displayed directly in the console.

No graphical interface or login system is used, which keeps the system simple and focused on the core operations needed to be accomplished.

6.2 Screen Images

Figure 1: Main Menu Screen

=== Employee Management System ===

1. Search Employee
2. Add Employee
3. Update Email/Phone
4. Increase Salary
5. Report by Division
6. Pay History
7. Full Update (Salary/Job/Division)
8. Report by Job Title
9. Exit

Figure 2: Search Employee Screen (e.g. Employee ID: 1)

Enter Employee ID, Name, or SSN: 1
--- Employee Info ---
ID: 1
Name: John Smith
SSN: 123456789
Email: john.smith@companyz.com
Phone: 555-100-1001
Salary: \$95000.0
Division: Engineering
Job Title: Software Engineer

Figure 3: Add Employee Screen (e.g. Frodo Baggins)

First Name: Frodo
Last Name: Baggins
SSN: 993239504
Email: frodoBaggins101@gmail.com
Phone: 145-999-4405
Hire Date (YYYY-MM-DD): 2026-04-25
Salary: 100000
JobTitleID: 6
DivisionID: 1
Employee added successfully.

Figure 4: Update Employee Screen (e.g. Employee: Jane Doe)

Employee ID: 2
New Email: jane.doe@company2.com

New Phone: 555-111-2223

Employee updated.

Figure 5: Increase Salary Screen

Min Salary: 50000

Max Salary: 85000

Percent ($0.032 = 3.2\%$): 0.032

10 employees updated.

Figure 6: Report by Division Screen

--- Pay by Division ---

Engineering: \$99937.5

Human Resources: \$36562.5

Marketing: \$43875.0

Finance: \$60937.5

Operations: \$34875.0

Figure 7: Pay Statement History (e.g. Employee ID: 1)

Enter Employee ID: 1

--- Pay History ---

Date: 2024-01-31 | Gross: \$7916.67 | Net: \$5937.5

Date: 2024-02-29 | Gross: \$7916.67 | Net: \$5937.5

Date: 2024-03-31 | Gross: \$7916.67 | Net: \$5937.5

Figure 8: Full Update Screen (e.g. Employee ID: 3)

Employee ID: 3

New Salary: 145000

New JobTitleID: 2

New DivisionID: 3

Employee updated (full).

Figure 9: Report by Job Title Screen

--- Pay by Job Title ---

Software Engineer: \$34312.5

HR Manager: \$34312.5

Marketing Specialist: \$26625.0

Accountant: \$26625.0

Operations Manager: \$34875.0

Junior Developer: \$24375.0
Senior Developer: \$21562.5
Financial Analyst: \$34312.5
HR Coordinator: \$21937.5
Marketing Director: \$17250.0

6.3 Screen Objects and Actions

Figure 1: Main Menu Screen

Menu Options (1–9): List of system functions.
Action: User selects a number to navigate.

Input Prompt: “Enter choice” field.
Action: Accepts user input and directs program flow.

Figure 2: Search Employee Screen (e.g. Employee ID: 1)

Search Input: Field to enter employee ID, name, or SSN.
Action: Used to search for employee in database.

Employee Info Display: Shows employee details.
Action: Displays retrieved employee information.

Figure 3: Add Employee Screen (e.g. Frodo Baggins)

Input Fields: Fields for name, SSN, email, phone, etc.
Action: Collects new employee data.

Confirmation Message: “Employee added successfully.”
Action: Confirms insertion into database.

Figure 4: Update Employee Screen (e.g. Employee: Jane Doe)

Employee ID Input: Identifies employee.
Action: Selects record to update.

Email Input: Field for new email.
Action: Updates email in database.

Phone Input: Field for new phone number.
Action: Updates phone in database.

Confirmation Message: Success message.

Action: Confirms update completed.

Figure 5: Increase Salary Screen

Min Salary Input: Lower bound of range.

Action: Filters employees.

Max Salary Input: Upper bound of range.

Action: Filters employees.

Percent Input: Salary increase percentage.

Action: Applies raise to employees.

Result Message: Displays number of updates.

Action: Confirms operation result.

Figure 6: Report by Division Screen

Division Name Output: Displays division names.

Action: Groups employees by division.

Total Pay Output: Shows total payroll per division.

Action: Displays aggregated salary data.

Figure 7: Pay Statement History

Employee ID Input: Field to enter employee ID.

Action: Retrieves pay history for the specified employee.

Pay History Display: Shows date, gross pay, and net pay per period.

Action: Displays all pay statement records for the employee.

Figure 8: Full Update Screen

Employee ID Input: Identifies employee.

Action: Selects record to update.

Salary Input: Field for new salary.

Action: Updates salary in database.

JobTitleID Input: Field for new job title ID.

Action: Updates job title in database.

DivisionID Input: Field for new division ID.

Action: Updates division in database.

Confirmation Message: "Employee updated (full)."

Action: Confirms update completed.

Figure 9: Report by Job Title Screen

Job Title Name Output: Displays job title names.

Action: Groups employees by job title.

Total Pay Output: Shows total payroll per job title.

Action: Displays aggregated pay data.

7.0 REQUIREMENTS MATRIX

Requirement ID	Requirement Description	Implemented In
R1	Add SSN column to employee table	MySQL Database, EmployeeDAO
R2	Search employee by Employee ID, Exact Full-Name, or SSN	Main, EmployeeDAO
R3	Add new employee	Main, EmployeeDAO
R4	Update employee contact (email, phone)	Main, EmployeeDAO
R5	Increase salary by percentage within range	Main, EmployeeDAO
R6	Generate total pay by division	Main, EmployeeDAO
R7	View pay statement history by employee	Main, EmployeeDAO
R8	Full update of salary, job title, division	Main, EmployeeDAO
R9	Generate total pay by job title report	Main, EmployeeDAO
R10	Store pay statement history	MySQL Database
R11	No authentication needed	System Design